VirtualNext Smart 开发平台使用说明

1. 平台概述

1.1. 平台说明

VirtualNext 开发平台基于 3D 建模和仿真技术打造，系统包含运动控制组件，相机组件及通信接口，开发者可根据需求定制视觉算法和控制逻辑。

1.2. 系统组成

VirtualNext 开发平台可仿真实物动作，包含物料生成、三轴伺服运动、相机取图、智能感知货架等功能。

VirtualNext 作为仿真开发平台，开发者可进行相关的视觉算法和运动控制逻辑的开发。开发包中已包含视觉处理系统框架与运动控制系统框架。系统之间通过 TCP socket 通讯。

在视觉处理系统框架中，开发者可集成自行开发的 AI 视觉处理算法以处理相机图片，并根据特征输出对应置物位置信息提供给运动控制系统；运动控制系统框架具备启停、取放、与视觉处理系统交互等功能。开发者也可以自定义控制逻辑，提升系统运行效率。

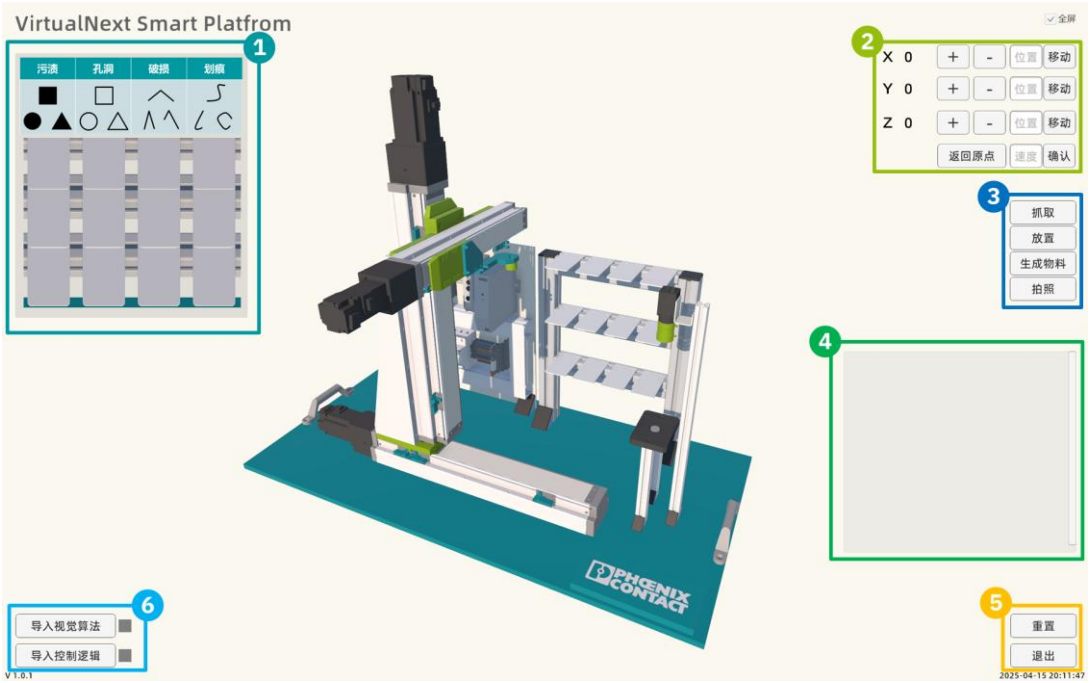
开发者可通过 VirtualNext 画面集中控制实现全自动运行。

2. VirtualNext Smart 开发平台

2.1. 平台介绍

2.1.1. 整体概述

VirtualNext 控制画面如下图所示。



以下是图中数字标记的各部分功能说明：

(1) 题目示例以及放料位结果展示。

(2) 手动调试：XYZ 轴坐标显示，“+”、“-”步进移动，输入“位置”值并点击“移动”按钮，可将对应轴移动至指定位置。输入“速度”并点击“确认”，可设定三轴的移动速度。点击“返回原点”按钮，可将三轴移动回原点。

(3) 手动调试：当三轴末端吸盘接触到物料时，点击“抓取”即可抓紧物料，点击“放置”释放物料，点击“生成物料”在置物台生成一个物料，点击“拍照”可捕捉置物台上图像（需已连接算法程序）。

(4) 显示运行日志信息用于调试。

(5) 点击“重置”清除所有物料并将三轴返回原位，点击“退出”退出程序。

(6) 导入视觉算法和控制逻辑并运行，并会提示连接状态。

2.2. 操作使用

2.2.1. 操作说明

按住鼠标右键并拖动以旋转视角；滑动鼠标滚轮可以改变视角远近。

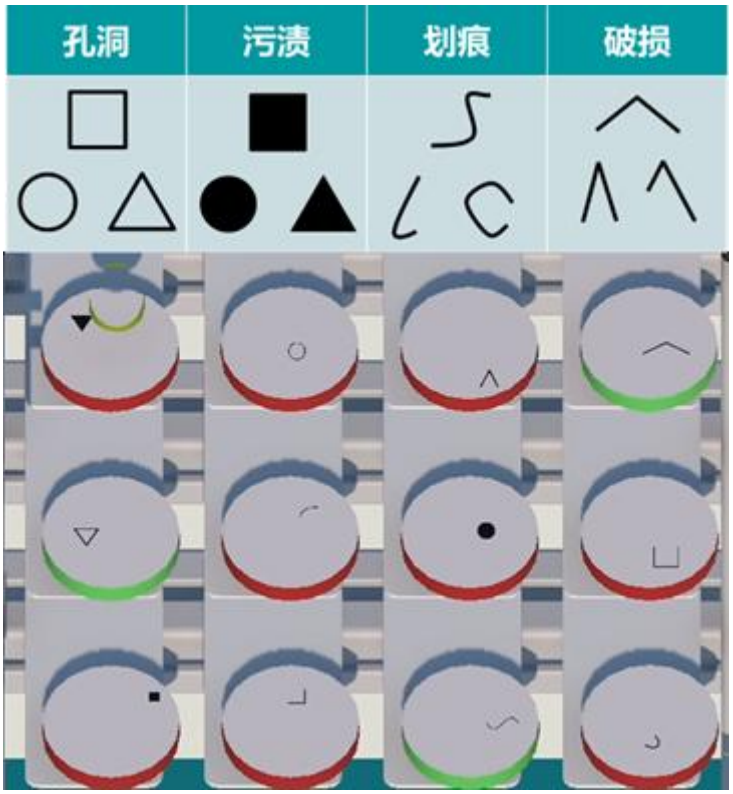
2.2.2. 物料摆放要求

物料放置要求为每个库位有且只有一个正确放置的物料。

如下图所示：共四大类缺陷。放置位四列对应四大类缺陷。每一列放置的物料位可以是该列三个位置中的任意一个，当放置不属于该列的缺陷物料时不计分，放置正确物料才能得分。

但是要注意，当一个放置位已有正确的物料后，不应再往上堆叠物料，后续无论放置何种物料均为错误，不得分。

当物料被放置到物料架后系统会自动判断得分情况，若放置正确，物料会变为绿色；放置错误，则变成红色。



2.2.3. 调试说明

下图为部分参数设置及控制界面。



X、Y、Z 显示当前位置信息，提供点动和连续移动两种示教方式，可以设置三轴的速度，设置范围为 1-5，X 轴移动区间为 0-300，Y 轴移动区间为 0-300，Z 轴移动区间为 0-200。若移动范围超出限制，系统会提示异常。注意：当控制程序运行时，请勿使用手动示教功能，以避免冲突。

三轴移动到抓取位后可以抓取物料，到达放置位后可以释放物料。物料有可能会产生堆叠，当堆叠过高时可能会出现意外情况。

连接至视觉算法程序后，点击拍照后会拍摄当前图片并直接发送到视觉算法程序进行处理，拍摄的图片会保存到程序目录下“received_images”文件夹中（该文件夹请勿修改），可以自行查看。下图为 3 张图片样例。



2.2.4. 使用步骤

按照上述调试说明设置好相关参数后，分别点击“导入视觉算法”和“导入控制逻辑”并选择相应程序即可自动建立连接。

3. 视觉处理系统

3.1. 系统介绍

3.1.1. 整体概述

系统通过 TCP 通信接收图像数据，并对图像进行处理分析，将结果返回给 PLC 控制系统。整个过程实现了自动化视觉检测的基本流程，涵盖通信、图像接收、图像处理与结果反馈四个主要环节。

系统使用 Python 开发，如需使用其它语言开发视觉处理算法，可参考模板自行实现通信等相关功能。

3.2. 系统使用

3.2.1. 通讯说明

系统包含两个通信端口：

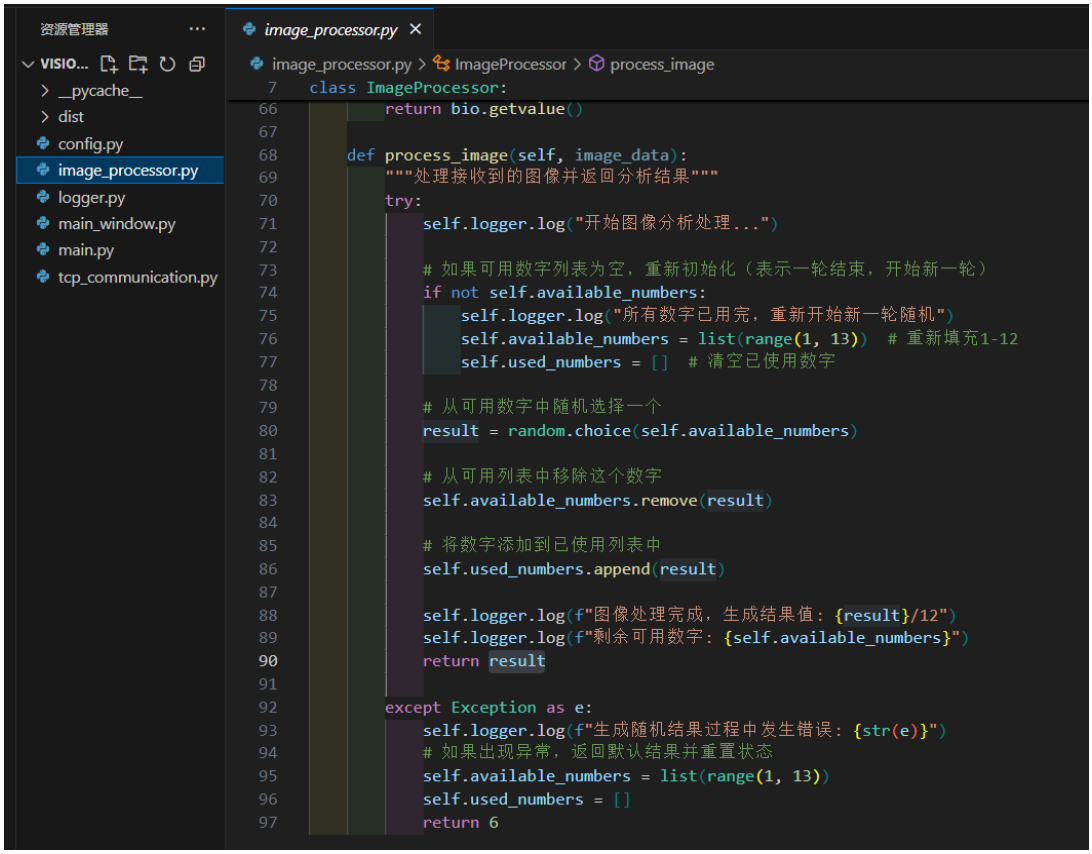
(1) Python 服务器端(用来与 PLC 客户端连接)：接收 PLC 的触发命令

(2) Python 客户端(用来与 VirtualNext 服务器连接)：接收图像数据

通信流程为：运动控制发送触发命令 → Python 转发到 VirtualNext 服务器中的相机 → VirtualNext 相机返回图像 → Python 处理图像 → 结果返回给 PLC。

3.2.2. 开发图像处理算法

开发自定义图像处理算法，只需修改 image_processor.py 文件中的 process_image 方法。该方法接收图像数据作为输入，返回一个整数作为处理结果。所有通信、显示相关的代码均已预设，开发者只需专注于图像处理算法本身。



3.2.3. 传输变量定义

通信协议使用简单的字符串格式：

(1) PLC 触发命令：Camera:Trigger=True

(2) VirtualNext Smart 触发命令：Camera:Start=True

(3) 图像处理结果返回格式为：Camera:Result={值}

错误代码定义：

(1) 图像不完整：-1

(2) 连接超时：-2

(3) 通信错误：-3

(4) 连接拒绝：-4

3.2.4. 运行方法

在 VirtualNext 开发平台导入视觉算法程序（或者直接运行自己在视觉开发框架中编

写的视觉检测算法）,出现如下界面：



如果 VirtualNext Smart 已经运行，VisionApp 程序加载完成后，将自动连接 VirtualNext Smart 服务器，开启监听运动控制系统连接。

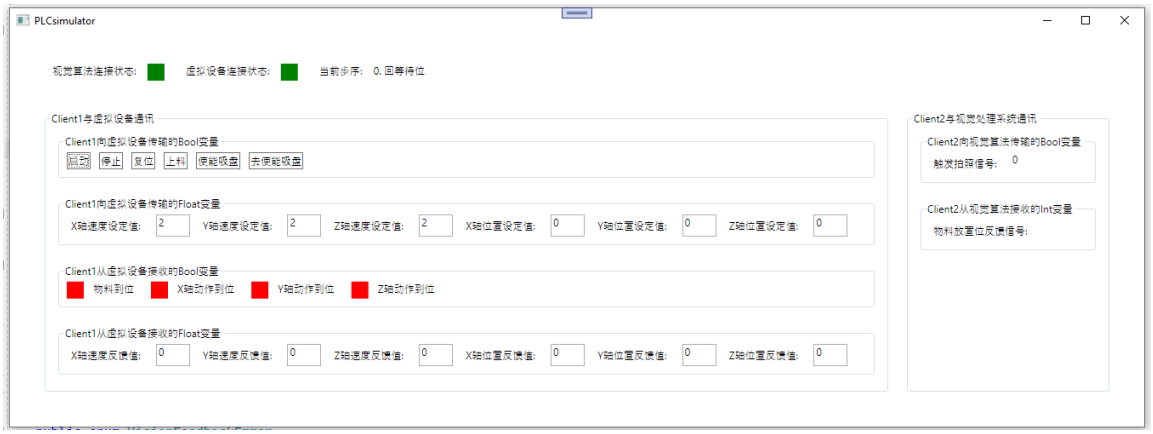
4. 运动控制系统

4.1. 系统介绍

4.1.1. 整体概述

本系统使用 C#开发，通过 TCP 通信驱动控制流程，如需使用其它语言编写控制逻辑，可参考模板自行实现通信等相关功能。

控制及状态显示界面如下图：



4.2. 系统使用

4.2.1. 通讯说明

系统采用 TCP Socket 通信方式，包含两组客户端：

(1) 作为客户端与 VirtualNext Smart 系统通信：控制伺服轴运动及 IO 信号交互。

(2) 作为客户端与视觉处理系统通信：触发拍照并接收视觉处理结果。

通信流程为：运动控制系统启动→运动控制系统上料→接收 VirtualNext 系统物料到位信号→触发视觉处理系统拍照检测→VirtualNext 系统抓取产品放置到指定位置。

4.2.2. 传输变量定义

(1) 伺服轴主要变量：

SpeedSetting：伺服轴速度设定值

PositionSetting：伺服轴目标位置设定值

Start：触发伺服轴运动

SpeedFeedback：伺服轴速度反馈值

PositionFeedback：伺服轴实际到达位置反馈值

Arrived：伺服轴是否已运动到位

(2) IO 主要变量：

MaterialFeed：上料信号

SuctionEnable：吸盘使能

SuctionEnabled：吸盘使能结果

MaterialInPosition：物料到位信号

4.2.3. 使用步骤

本程序实现的主要流程：

(1) 运行程序

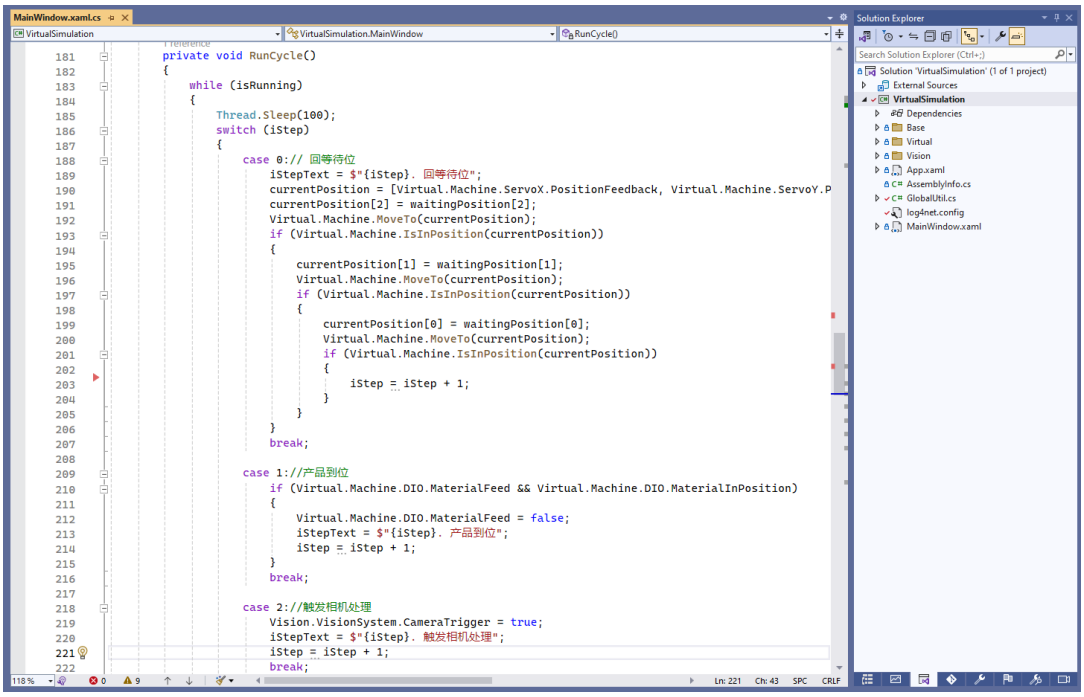
(2) 点击“启动”按钮

(3) 点击“上料”按钮

(4) 收到 VirtualNext 系统物料到位信号，界面中“物料到位”显示绿色

(5) “当前步序:”会实时显示当前流程步序

运动控制系统界面可实时显示与 VirtualNext 系统及视觉处理系统之间的通信变量状态。如需自行设计流程，可在主窗体后台程序的 RunCycle()方法中修改。



5. 总结

5.1. 使用总结

在完成视觉处理系统、运动控制系统开发及上述配置后，开发者可通过 VirtualNext 画面一键启动实现仿真系统全自动运行。

5.2. 注意事项

系统运行时需检查防火墙相关设置，避免端口禁用影响程序运行。